PROJECT WORK

Mini Bank Account Management System

Python Console Application

Name	**Raunak Singh**
Registration No.	26BCE10537
Branch	CSE
Programme	CSE Core
Academic Year	2026–27

Submitted as an academic programming project

DECLARATION

I, **Raunak Singh**, Registration No. **26BCE10537**, student of **CSE Core**, declare that this project titled **Mini Bank Account Management System** has been developed as an academic programming project using Python. The project demonstrates functions, dictionaries, loops, conditional statements, input validation and exception handling.

Student Name	**Raunak Singh**
Registration No.	26BCE10537
Academic Year	2026–27
Signature	_____

1. ABSTRACT

The Mini Bank Account Management System is a menu-driven Python application that simulates basic banking operations. It allows a user to create an account, deposit money, withdraw money, check the current balance, and display account details. Account information is maintained in a Python dictionary while the program is running. Basic validation and exception handling are used to handle invalid input and unsafe transactions.

2. AIM

To develop a simple console-based bank account management system in Python that performs common account operations using fundamental programming concepts.

3. OBJECTIVES

1. Create and store bank account information.
2. Deposit money into an existing account.
3. Withdraw money when sufficient balance is available.
4. Check and display the current account balance.
5. Display account number, account holder name and balance.
6. Validate user input and handle common input errors.
7. Apply functions, dictionaries, loops and conditional statements.

4. TECHNOLOGIES AND CONCEPTS USED

Technology / Concept	Purpose
Python	Programming language used to implement the system.
Dictionary	Stores account number, holder name and balance.
Functions	Separates operations into reusable modules.
while loop	Keeps the main menu running until Exit.
if / elif / else	Controls menu choices and validation.
try / except	Handles invalid numeric input.
User input	Collects account and transaction information.

5. SYSTEM DESIGN / WORKING

The program starts with an empty **accounts** dictionary. The **main()** function repeatedly displays a menu. According to the user's choice, it calls the required function. Each operation checks whether the account exists and validates the entered amount before changing the balance.

Program Flow

Step	Operation
1	Start the program.
2	Display the main menu.
3	Read the user's choice.
4	Call Create, Deposit, Withdraw, Check Balance or Display Details.
5	Validate account number and amount where required.
6	Update or display account information.
7	Repeat until the user selects Exit.

Data Representation

Account data is stored in a nested dictionary. The account number is the main key, while the holder name and balance are stored as fields.

```
accounts = {
    "1001": {
        "name": "Raunak",
        "balance": 6000
    }
}
```

Main Functions

Function	Purpose
create_account()	Creates a new account after validating the initial deposit.
deposit()	Adds a valid deposit amount to an existing account.
withdraw()	Subtracts a valid amount if sufficient balance is available.
check_balance()	Displays the current balance of an existing account.
display_account()	Displays account number, holder name and balance.
main()	Controls the menu and calls the selected operation.

6. MODULE-WISE DESCRIPTION

6.1 Create Account

Asks for account number, holder name and initial deposit. Duplicate account numbers are rejected and negative/invalid initial deposits are not accepted.

6.2 Deposit Money

Checks account existence, accepts only a positive amount and adds it to the balance.

6.3 Withdraw Money

Checks account existence, validates the amount and prevents withdrawal above the available balance.

6.4 Check Balance

Searches for the account and displays its current balance.

6.5 Display Account Details

Displays account number, account holder name and current balance.

6.6 Exit

Selecting option 6 breaks the menu loop and terminates the program.

7. INPUT VALIDATION AND ERROR HANDLING

The program uses **try-except** while converting monetary input to float. It also checks duplicate accounts, missing accounts, negative initial deposits, non-positive deposits or withdrawals, and insufficient balance.

8. ALGORITHM

```
1. Start.
2. Initialize an empty accounts dictionary.
3. Display the banking menu.
4. Read the user's choice.
5. If choice is 1, create an account.
6. If choice is 2, deposit money.
7. If choice is 3, withdraw money.
8. If choice is 4, check balance.
9. If choice is 5, display account details.
10. If choice is 6, display exit message and stop.
11. Otherwise, display an invalid-choice message.
12. Repeat steps 3–11 until the user exits.
```

9. SAMPLE EXECUTION / OUTPUT

The following representative run demonstrates account creation, deposit, withdrawal, balance checking and account-detail display.

```
=====================================
   MINI BANK ACCOUNT MANAGEMENT SYSTEM
=====================================
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Check Balance
5. Display Account Details
6. Exit
Enter your choice: 1

--- Create Account ---
Enter account number: 1001
Enter account holder name: Raunak
Enter initial deposit: 5000
Account created successfully!

Enter your choice: 2

--- Deposit Money ---
Enter account number: 1001
Enter amount to deposit: 2000
Money deposited successfully.
Current balance: 7000.0

Enter your choice: 3

--- Withdraw Money ---
Enter account number: 1001
Enter amount to withdraw: 1000
Money withdrawn successfully.
Current balance: 6000.0

Enter your choice: 4

--- Check Balance ---
Enter account number: 1001
Current balance: 6000.0

Enter your choice: 5

--- Account Details ---
Enter account number: 1001
Account Number : 1001
Account Holder : Raunak
Balance        : 6000.0
```

10. TEST CASES

Test Case	Condition	Expected Result
Create account	New account + valid deposit	Account created successfully
Duplicate account	Existing account number	Account already exists
Deposit	Positive amount	Balance increases
Invalid deposit	Zero / negative amount	Deposit rejected
Withdraw	Amount ≤ balance	Balance decreases
Insufficient funds	Amount > balance	Withdrawal rejected
Unknown account	Non-existing account	Account not found
Invalid amount	Text instead of number	Valid amount message

11. SOURCE CODE

The source code below is the code used in the submitted **Account.py** project file.

```
accounts = {}


def create_account():
    print("\n--- Create Account ---")

    account_number = input("Enter account number: ")

    if account_number in accounts:
        print("Account already exists.")
        return

    name = input("Enter account holder name: ")

    try:
        initial_deposit = float(input("Enter initial deposit: "))

        if initial_deposit < 0:
            print("Initial deposit cannot be negative.")
            return

    except ValueError:
        print("Please enter a valid amount.")
        return

    accounts[account_number] = {
        "name": name,
        "balance": initial_deposit
    }

    print("Account created successfully!")


def deposit():
    print("\n--- Deposit Money ---")

    account_number = input("Enter account number: ")

    if account_number not in accounts:
        print("Account not found.")
        return

    try:
        amount = float(input("Enter amount to deposit: "))

        if amount <= 0:
            print("Deposit amount must be greater than zero.")
            return

    except ValueError:
        print("Please enter a valid amount.")
        return

    accounts[account_number]["balance"] += amount

    print("Money deposited successfully.")
    print("Current balance:", accounts[account_number]["balance"])


def withdraw():
    print("\n--- Withdraw Money ---")

    account_number = input("Enter account number: ")

    if account_number not in accounts:
        print("Account not found.")
        return

    try:
        amount = float(input("Enter amount to withdraw: "))

        if amount <= 0:
            print("Withdrawal amount must be greater than zero.")
            return

    except ValueError:
        print("Please enter a valid amount.")
        return

    if amount > accounts[account_number]["balance"]:
        print("Insufficient balance.")
```

```
        return

    accounts[account_number]["balance"] -= amount

    print("Money withdrawn successfully.")
    print("Current balance:", accounts[account_number]["balance"])


def check_balance():
    print("\n--- Check Balance ---")

    account_number = input("Enter account number: ")

    if account_number not in accounts:
        print("Account not found.")
        return

    print("Current balance:",
          accounts[account_number]["balance"])


def display_account():
    print("\n--- Account Details ---")

    account_number = input("Enter account number: ")

    if account_number not in accounts:
        print("Account not found.")
        return

    account = accounts[account_number]

    print("\nAccount Number :", account_number)
    print("Account Holder :", account["name"])
    print("Balance        :", account["balance"])


def main():

    while True:

        print("\n========================================")
        print("   MINI BANK ACCOUNT MANAGEMENT SYSTEM")
        print("========================================")
        print("1. Create Account")
        print("2. Deposit Money")
        print("3. Withdraw Money")
        print("4. Check Balance")
        print("5. Display Account Details")
        print("6. Exit")

        choice = input("Enter your choice: ")

        if choice == "1":
            create_account()

        elif choice == "2":
            deposit()

        elif choice == "3":
            withdraw()

        elif choice == "4":
            check_balance()

        elif choice == "5":
            display_account()

        elif choice == "6":
            print("Thank you for using the Bank Account Management System.")
            break

        else:
            print("Invalid choice. Please try again.")


main()
```

12. CONCLUSION

The Mini Bank Account Management System demonstrates how fundamental Python concepts can be combined to build a practical menu-driven application. It supports account creation, deposits, withdrawals, balance checking and account-detail display. Validation and exception handling improve the reliability of user interaction.

13. FUTURE SCOPE

1. Permanent storage using a database or file.
2. Customer login and PIN authentication.
3. Transaction history.
4. Fund transfer between accounts.
5. Account deletion and update features.
6. Graphical User Interface (GUI).
7. Different account types such as savings and current accounts.

14. REFERENCES

1. Python documentation and standard programming concepts.
2. Class notes and laboratory exercises on Python programming.

— End of Project Work —